

# INTERNSHIP PROJECT BRIEF

## Student Internship Management System

Spring Boot Web Application

|                       |                                                       |
|-----------------------|-------------------------------------------------------|
| Programme             | Software and Internet Technologies (SIT), Year 3      |
| Recommended team size | 3-4 students                                          |
| Suggested duration    | 6 weeks                                               |
| Project language      | English (documentation, source code and presentation) |

*Purpose: build and demonstrate a maintainable Spring Boot application using realistic business rules, teamwork and version control.*

# 1. Project Overview

Develop a web-based system that supports the complete process of publishing internship positions and applying for them. The system must allow students to browse opportunities, submit applications and track their status. Companies or mentors manage their own internship offers and applications. Administrators manage the entire platform.

## 1.1 Learning Objectives

- Design a small but complete web application with Spring Boot.
- Apply layered architecture: Controller -> Service -> Repository.
- Model relational data with Spring Data JPA and a SQL database.
- Implement REST endpoints, validation and centralized error handling.
- Apply authentication and authorization with user roles.
- Collaborate through Git and GitHub using branches, commits and pull requests.
- Document and demonstrate a working software product.

## 1.2 User Roles

| Role             | Main Responsibilities                                               | Access Scope                                       |
|------------------|---------------------------------------------------------------------|----------------------------------------------------|
| Student          | Create profile, browse offers, apply, track applications.           | Own profile and own applications.                  |
| Company / Mentor | Create and manage internship offers; review candidate applications. | Own company, own offers and related applications.  |
| Administrator    | Manage companies, users, offers and overall system status.          | All application data and administration functions. |

## 1.3 Typical Workflow

1. An administrator registers or approves a company profile.
2. A company publishes an internship offer.
3. A student searches available offers and submits an application.
4. The company reviews the application and changes its status.
5. The student sees the current application status in the personal dashboard.
6. The administrator can review platform statistics and manage inactive or invalid content.

# 2. Functional Requirements

## 2.1 Mandatory Requirements

### A. Authentication and User Profiles

- Registration and login for users.
- At least three roles: ADMIN, STUDENT and COMPANY.
- Role-based authorization: users must not access functions or data belonging to other roles.
- Student profile with faculty number, specialty, course, skills and contact details.
- Company profile with name, description, website, city and contact email.

### B. Internship Offers

- Company users can create, edit, close and view their own internship offers.

- Each offer must include title, description, required skills, location, type and application deadline.
- Students can view active offers and search by keyword.
- Students can filter offers by city, work type and technology/skill.
- An administrator can view all offers and deactivate an offer when needed.

### C. Applications

- A student can apply for an active internship offer.
- A student can apply only once for the same offer.
- An application must include date, motivation letter and status.
- Suggested statuses: PENDING, UNDER\_REVIEW, APPROVED, REJECTED.
- Students can see only their own applications.
- Companies can see applications only for their own offers.
- Companies can change the application status and optionally add a comment.

### D. Administration and Basic Reports

- Administrator can view and manage companies, offers and applications.
- Provide a simple dashboard with: total active offers, total applications, approved applications and most frequently requested skills or technologies.

## 2.2 Optional Extensions (Bonus)

- Upload a CV in PDF format.
- Send an email notification when an application status changes.
- JWT-based authentication.
- Pagination, sorting and advanced filtering.
- Audit log of status changes and administrative actions.
- Docker Compose configuration for application and database.
- Automated tests for critical business rules.

## 3. Technical Requirements

### 3.1 Required Technology Stack

| Area              | Requirement                           |
|-------------------|---------------------------------------|
| Language          | Java 17 or newer                      |
| Backend           | Spring Boot 3+                        |
| Web layer         | Spring Web / REST API                 |
| Persistence       | Spring Data JPA + MySQL or PostgreSQL |
| Validation        | Jakarta Bean Validation               |
| Security          | Spring Security with roles            |
| Build             | Maven or Gradle                       |
| API documentation | Swagger / OpenAPI                     |
| Version control   | Git and GitHub                        |

### 3.2 Recommended Project Structure

```
src/main/java/...  
  controller/  
  service/  
  repository/  
  entity/  
  dto/  
  mapper/  
  exception/  
  security/  
  config/
```

Business logic must be placed in the service layer. Controllers should only receive requests, validate input, call services and return responses.

### 3.3 Minimum Data Model

| Entity          | Minimum Fields                                                           | Key Relations                                          |
|-----------------|--------------------------------------------------------------------------|--------------------------------------------------------|
| User            | id, firstName, lastName, email, password, role, createdAt                | One user may have one student or company profile.      |
| StudentProfile  | id, facultyNumber, specialty, course, skills                             | Belongs to one User; submits many Applications.        |
| Company         | id, name, description, website, contactEmail, city                       | Belongs to one User; owns many InternshipOffers.       |
| InternshipOffer | id, title, description, requiredSkills, location, type, deadline, status | Belongs to one Company; has many Applications.         |
| Application     | id, applicationDate, motivationLetter, status, comment                   | Belongs to one StudentProfile and one InternshipOffer. |

### 3.4 Minimum REST Endpoints

```
POST /api/auth/register  
POST /api/auth/login  
GET /api/internships  
GET /api/internships/{id}  
POST /api/internships  
PUT /api/internships/{id}  
DELETE /api/internships/{id}  
GET /api/companies  
POST /api/companies  
POST /api/applications  
GET /api/applications/my  
GET /api/internships/{id}/applications  
PATCH /api/applications/{id}/status  
GET /api/admin/statistics
```

## 4. Mandatory Git and GitHub Requirements

Every team must create one shared private or public GitHub repository before coding begins. The repository is part of the assessment and will be reviewed during the final defense.

### 4.1 Repository Setup

- Repository name format: sit-internship-management-team-XX.

- All team members and the supervisor/teacher must be added as collaborators.
- The repository must contain a meaningful README.md from the first week.
- Add a correct .gitignore for Java, IntelliJ IDEA/Eclipse and environment-specific files.
- Do not commit passwords, JWT secrets, database credentials or local configuration files.
- Use application.properties.example or application.yml.example for configuration examples.

## 4.2 Required README.md Content

- Project title and short description.
- Team members and individual responsibilities.
- Technology stack.
- Instructions for starting the application and database.
- Database configuration example.
- API documentation link or Swagger endpoint.
- Known limitations and optional features implemented.

## 4.3 Workflow Rules

7. The main branch must always remain runnable and stable.
8. Every new feature or bug fix must be developed in a separate branch.
9. Use descriptive branch names, for example: feature/student-registration, feature/application-status, fix/duplicate-application.
10. Each feature must be merged into main through a pull request.
11. At least one teammate must review the pull request before merging whenever possible.
12. Resolve merge conflicts together; do not delete another student's work to solve a conflict.
13. Every member must have visible, meaningful contributions in the repository history.

## 4.4 Commit Standards

Commit messages must describe the actual change. Avoid messages such as “update”, “work”, “final”, “test”, or “asdf”.

Good examples:

```
feat: add internship offer creation endpoint
fix: prevent duplicate student applications
docs: add setup instructions to README
test: add service tests for application status update
refactor: move validation logic to application service
```

- Make small and logical commits. Do not commit the entire project only at the end.
- Each student should normally have at least 8-10 meaningful commits during the project.
- Commit source code, documentation and configuration changes separately when practical.
- Do not commit generated folders such as target/, build/, .idea/ or node\_modules/.

## 4.5 Suggested Git Commands

```
git clone <repository-url>
git checkout -b feature/student-registration
git status
git add .
git commit -m "feat: add student registration"
git push -u origin feature/student-registration
# Create a Pull Request in GitHub
git checkout main
git pull origin main
```

## 5. Project Milestones and Deliverables

| Week | Deliverable                              | Minimum Evidence                                                           |
|------|------------------------------------------|----------------------------------------------------------------------------|
| 1    | Planning and repository setup            | GitHub repository, README, team roles, use-case diagram, draft ER diagram. |
| 1    | Data layer and basic CRUD                | Entities, relations, repositories, database schema, initial endpoints.     |
| 2    | Service layer and validation             | DTOs, mapping, validation, centralized exception handling.                 |
| 2    | Authentication and permissions           | Security configuration, roles and access restrictions.                     |
| 3    | Applications and user interface/demo     | Core workflow completed; Swagger and/or a simple web interface.            |
| 3    | Testing, documentation and final defense | Final README, API documentation, tests, presentation and demonstration.    |

## 6. Final Submission

- GitHub repository link.
- Final source code in the repository.
- README.md with full setup instructions.
- Database schema or ER diagram.
- Use-case diagram and short architecture diagram.
- Swagger/OpenAPI documentation or Postman collection.
- Presentation in English (approximately 8-12 slides).
- Live demonstration of the main user workflow.
- Short individual explanation by each team member of their own contribution.

## 7. Assessment Criteria (100 points)

| Criterion                     | What will be assessed                                                | Points |
|-------------------------------|----------------------------------------------------------------------|--------|
| Core functionality            | Working offers, applications, user roles and main workflow.          | 30     |
| Architecture and code quality | Layered design, DTOs, readable code, separation of responsibilities. | 15     |
| Database design               | Correct entities, relationships, constraints and persistence.        | 15     |
| Security and validation       | Roles, access control, input validation and error handling.          | 10     |
| Documentation and API quality | README, Swagger/Postman, diagrams and setup instructions.            | 10     |
| Git and teamwork              | Branches, commits, pull requests, contribution traceability.         | 10     |
| Individual defense            | Ability to explain personal code and decisions.                      | 10     |

## 8. Final Submission Checklist

*Before submitting, the team must verify the following items:*

- ☐ The application starts successfully on another computer using the instructions in README.md.
- ☐ The database schema and sample data (if used) are available or clearly described.
- ☐ All mandatory roles and core workflows have been demonstrated.
- ☐ Swagger/OpenAPI or a Postman collection is available.
- ☐ The main branch contains the final, runnable version of the application.
- ☐ Every student has visible commits and can explain their individual contribution.
- ☐ No passwords, private keys, database credentials or generated build folders are committed.
- ☐ The presentation and the live demonstration are prepared in English.

## 9. Important Notes

- The project must be understandable, runnable and demonstrable. A smaller completed system is better than a large unfinished system.
- Use clean naming in English for classes, methods, variables, database fields and API endpoints.
- External libraries and online examples may be used only when understood and referenced where appropriate.
- Every student must be able to explain the code they contributed during the final defense.
- The supervisor may ask any team member to demonstrate a specific feature or explain a Git commit.

## 10. Individual Contribution Statement

Each team member must add a short statement in the README.md or in a separate file named CONTRIBUTIONS.md. The statement must identify the implemented modules, significant commits and the technical decisions for which the student was responsible.

**Success criterion: the project should demonstrate a realistic Spring Boot application, disciplined use of Git and the ability to work as a software development team.**